**UNIVERSITÀ DI BOLOGNA**

Corso di Laurea Magistrale in Ingegneria Informatica

# Attività Progettuale in Fondamenti di Intelligenza Artificiale

## Classificazione di Immagini tramite Genetic Programming

**Studente:** Daniele Guiducci
**Docente:** Prof. Andrea Roli

**Anno Accademico:** 2024–2025

# Indice

**1 Introduzione e Obiettivi** **2**

**2 Descrizione del Dataset** **3**

**3 Estrazione delle Feature** **5**
3.1 Pipeline di estrazione . . . . . 5
3.2 Pseudocodice dell'estrazione . . . . . 6
3.3 Normalizzazione e generazione del file `problem.dat` . . . . . 6

**4 Algoritmo Genetico** **7**
4.1 Rappresentazione degli individui . . . . . 7
4.2 Calcolo della fitness . . . . . 8
4.3 Generazione della popolazione iniziale . . . . . 8
4.4 Ciclo evolutivo . . . . . 9
4.5 Esecuzione dell'algoritmo . . . . . 10

**5 Classificazione e Valutazione dei Risultati** **11**
5.1 Classificazione di un singolo esempio . . . . . 11
5.2 Classificazione del dataset di test . . . . . 12
5.3 Valutazione delle prestazioni . . . . . 12

**6 Conclusioni** **13**

# Capitolo 1

# Introduzione e Obiettivi

In questa attività progettuale è stato sviluppato un classificatore per immagini sintetiche, basato su un modello di Genetic Programming. L'obiettivo principale è la distinzione tra due classi di figure geometriche: **cerchi** e **quadrati**, generati artificialmente con diverse dimensioni, posizioni e rotazioni. Il compito dell'algoritmo evolutivo è quello di evolvere autonomamente una funzione che partendo da features delle immagini in input riesca a restituire un valore numerico interpretabile come una classificazione binaria.

$$f(a, b, c, d, ...) \rightarrow \{0, 1\}$$

dove il valore 0 identifica un cerchio e il valore 1 un quadrato e a,b,c,d sono le features quantitative estratte dalle immagini del dataset.

Il progetto si è articolato in quattro fasi principali:

- Creazione del dataset di immagini (cerchi e quadrati, con possibili rotazioni e casi tagliati).
- Estrazione delle feature dalle immagini per generare il file `.dat` di input per TinyGP.
- Riscrittura e adattamento di TinyGP in Java, facendo riferimento al paper di base.
- Test del modello sulle immagini di test e calcolo dell'accuracy.

Lo scopo del progetto è quello di analizzare la capacità del Genetic Programming di scoprire autonomamente relazioni tra le features delle immagini, producendo formule interpretabili che permettono di classificare nuove immagini in modo molto rapido, essendo basate esclusivamente su operazioni matematiche di base (ADD, MUL, SUB, DIV).

# Capitolo 2

# Descrizione del Dataset

La prima fase del progetto si è basata nella generazione del dataset fatto di immagini sintetiche di cerchi e quadrati neri con sfondo bianco. Per evitare che il modello imparasse regole banali come dimensione e posizione fissa, si è scelto di introdurre un certo grado di variabilità nella generazione delle figure:

- Per i cerchi: centro e raggio variabili
- Per i quadrati: centro variabile e rotazione (tra 0° e 90°) di almeno il 50% delle figure

**Osservazione:** si è notato che se la figura viene troppo tagliata (centro della figura vicino al bordo dell'immagine) ci sono dei problemi nella classificazione finale.

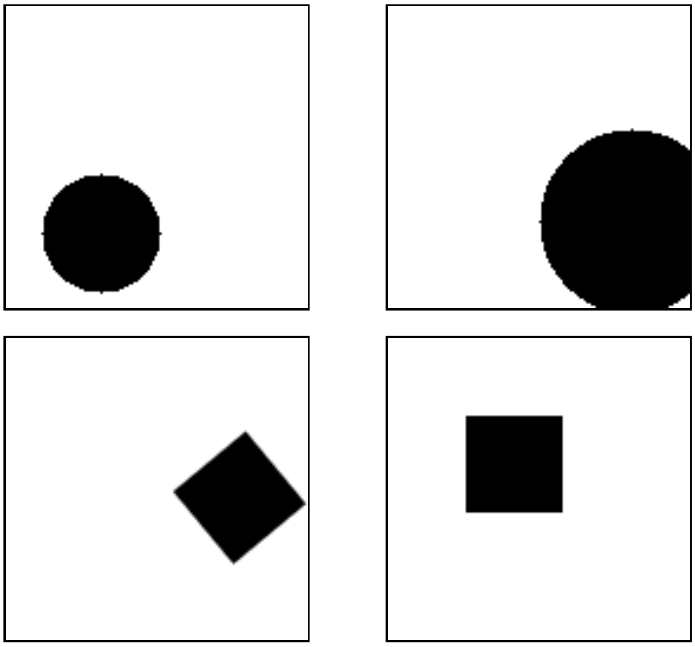


Figura 2.1: Esempi di immagini del dataset: cerchi e quadrati generati artificialmente con OpenCV.

Il dataset complessivo è composto da 1000 immagini (700 per training e 300 per testing), equamente bilanciato in cerchi e quadrati; in seguito è riportato lo pseudocodice delle funzioni usate per generare il dataset:

**Algorithm 1** Generazione di un'immagine con un cerchio

```
Crea un'immagine bianca di dimensione N × N (N=128)
Scegli casualmente il centro (x_c, y_c) all'interno di un intervallo (intervallo [20, 108])
Estrai casualmente il raggio r da un intervallo predefinito (intervallo [10, 50])
Disegna un cerchio nero pieno di centro (x_c, y_c) e raggio r
return immagine
```

**Algorithm 2** Generazione di un'immagine con un quadrato

```
Crea un'immagine bianca di dimensione N × N
Estrai casualmente la posizione (x, y) del vertice top-left (intervallo [10, 90])
Estrai casualmente la lunghezza del lato s (intervallo [20,40])
Disegna un quadrato nero con vertice (x, y) e lato s
if n° casuale < 0.5 then
    Estrai un angolo di rotazione θ ∈ [0°, 90°]
    Calcola la matrice di rotazione centrata nell'immagine
    Applica la rotazione all'immagine
end if
return immagine
```

# Capitolo 3

# Estrazione delle Feature

## 3.1 Pipeline di estrazione

Per rendere compatibili le immagini del dataset con l'algoritmo evolutivo, è necessario estrarre da ciascuna immagine delle feature numeriche che verranno fornite in input al GP per modellare la funzione di classificazione.

Per ogni immagine (cerchio o quadrato) sono stati eseguiti i seguenti passi:

1. Conversione in scala di grigi
2. Binarizzazione tramite soglia fissa: viene applicata una soglia per distinguere nettamente la figura nera dallo sfondo.
3. Estrazione del contorno tramite `findContours`: per calcolare proprietà geometriche.
4. Calcolo di misure geometriche derivate dal contorno: feature quantitative riportate in seguito.
5. Memorizzazione delle feature e dell'etichetta della classe: salvate in un file `.dat` per effettuare il training del modello.

Tutte le operazioni sono state eseguite con la libreria OpenCV in Python. Le feature estratte sono semplici da calcolare e discriminative per distinguere cerchi e quadrati.

Le quattro feature calcolate per ogni immagine sono:

- **Area** del contorno principale
- **Perimetro** della figura
- **Aspect ratio** (rapporto larghezza/altezza del bounding box): circa 1 per cerchi e quadrati non ruotati, si discosta da 1 per quadrati ruotati

$$\mathrm{AR} = \frac{\text{larghezza}}{\text{altezza}}$$

- **Extent** (rapporto tra area della figura e area del bounding box), indicatore di quanto la figura riempie lo spazio minimo che la contiene

$$\mathrm{Ext} = \frac{\mathrm{Area}}{w \cdot h}$$

## 3.2 Pseudocodice dell'estrazione

Di seguito è riportato lo pseudocodice che descrive il processo implementato:

**Algorithm 3** Estrazione delle feature da un'immagine

```
Carica l'immagine
Applica una treshold per ottenere l'immagine binaria
Trova i contorni esterni della figura
if nessun contorno trovato then
    scarta l'immagine
end if
Seleziona il contorno principale
Calcola l'area del contorno
Calcola il perimetro
Determina il rettangolo di bounding (x, y, w, h)
Calcola l'aspect ratio w/h
Calcola l'extent area/(w·h)
Restituisci il vettore delle feature
```

## 3.3 Normalizzazione e generazione del file `problem.dat`

Il modello `TinyGP` richiede in input un file `.dat` contenente i valori di tutte le feature normalizzati in un intervallo stabilito, insieme all'etichetta della classe. La normalizzazione è necessaria perché le quattro feature hanno ordini di grandezza diversi (ad esempio l'area può valere centinaia di pixel, mentre l'aspect ratio è compreso tipicamente tra 0.7 e 1.3).
La normalizzazione garantisce che tutte le feature contribuiscano in modo equilibrato alla funzione da evolvere, migliorando la convergenza del GP.
Per ogni feature è stata applicata una normalizzazione Min-Max:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

Il file finale `problem.dat` segue la struttura:

- prima riga: numero di variabili, intervallo dei valori e numero dei casi
- righe successive: quattro feature normalizzate seguite dall'etichetta della classe

```
problem.dat
4 10 -2 2 700
0.38 0.53 0.52 0.54 0.00
0.28 0.43 0.52 0.52 0.00
0.89 0.93 0.52 0.57 0.00
0.35 0.50 0.52 0.53 0.00
```

Figura 3.1: Righe del problem.dat

# Capitolo 4

# Algoritmo Genetico

In questo capitolo viene descritto il funzionamento dell'algoritmo utilizzato per evolvere automaticamente una funzione in grado di distinguere cerchi e quadrati a partire dalle quattro feature estratte. L'implementazione è basata su *Tiny-GP*, una versione minimale e compatta della Programmazione Genetica (Genetic Programming, GP) sviluppata in Java per motivi di efficienza.

## 4.1 Rappresentazione degli individui

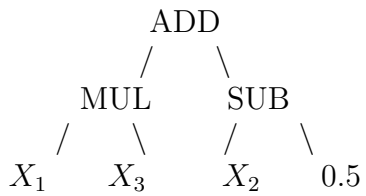
Ogni individuo della popolazione rappresenta un programma in forma prefissa, memorizzato come un array di caratteri. Il linguaggio per la costruzione degli alberi sintattici è composto da:

- **Terminali**: le variabili in ingresso (X1, X2, ..., Xn) e costanti casuali;
- **Funzioni**: operatori aritmetici `ADD`, `SUB`, `MUL`, `DIV`. L'operatore di divisione è protetto, evitando divisioni per valori prossimi a zero.

Ogni individuo è quindi un albero in cui i nodi interni sono operatori e le foglie sono variabili o costanti. Ogni individuo della popolazione può essere rappresentato come un programma in forma prefissa.
Ad esempio, un individuo può essere codificato come:

ADD MUL X1 X3 SUB X2 0.5

che corrisponde all'albero sintattico:




La funzione rappresentata dall'individuo è quindi:

$$f(X_1, X_2, X_3) = X_1 \cdot X_3 + (X_2 - 0.5)$$

## 4.2 Calcolo della fitness

La funzione di fitness utilizzata in TinyGP misura la capacità di un individuo di classificare correttamente le immagini del dataset. Ogni individuo, rappresentato come un programma prefisso, prende in input le feature normalizzate di un'immagine $(X_1, X_2, \ldots, X_n)$ e produce un valore numerico continuo. Questo valore viene poi trasformato tramite la funzione sigmoide:

$$\hat{y} = \frac{1}{1 + e^{-f(X_1, X_2, \ldots, X_n)}}$$

per ottenere un output compreso tra 0 e 1, interpretabile come probabilità di appartenere alla classe "quadrato" (o "cerchio"). La fitness dell'individuo è calcolata come la somma degli errori assoluti rispetto ai target desiderati su tutti i casi di training:

$$\text{fitness} = -\sum_{i=1}^{N} \left| \hat{y}_i - y_i \right|$$

dove $y_i$ rappresenta l'etichetta corretta dell'immagine $i$. In questo modo, più la funzione prodotta dall'individuo si avvicina ai valori attesi, minore è l'errore e maggiore è la fitness (ricordando che viene restituito il valore negativo dell'errore totale per uniformarsi alla logica di massimizzazione).

**Nota:** inizialmente è stato introdotto un termine di penalità proporzionale alla lunghezza del programma per limitare la crescita eccessiva delle formule, riducendo la complessità dei programmi generati. Questo tentativo non è stato incluso nella versione finale, poiché TinyGP, pur generando alcune formule molto lunghe, ha comunque mostrato ottime prestazioni di classificazione, inoltre con la penalizzazione a volte l'algoritmo non converge.

## 4.3 Generazione della popolazione iniziale

La popolazione iniziale viene generata attraverso l'operatore `grow`, che costruisce ricorsivamente alberi casuali fino a una profondità massima prefissata. Per ogni individuo:

- viene generato casualmente l'albero sintattico;
- ne viene calcolata immediatamente la fitness.

L'evoluzione degli individui nel modello `TinyGP` avviene tramite tre operatori fondamentali: **selezione**, **crossover** e **mutazione**.

- **Selezione (Tournament Selection)** La selezione è basata su tornei tra un numero ristretto di individui (`TSIZE`). Vengono utilizzate due varianti:
  - *selezione positiva*: tra i candidati si sceglie il migliore, che fungerà da genitore;
  - *selezione negativa*: tra i candidati si sceglie il peggiore, che verrà sostituito dal nuovo individuo generato.

- **Crossover** Serve a combinare porzioni di due individui per creare un nuovo programma. Il procedimento è:
  1. si seleziona un punto casuale nel primo genitore e si identifica il sottoalbero corrispondente;
  2. si ripete la stessa operazione nel secondo genitore;
  3. si costruisce il nuovo individuo sostituendo il sottoalbero del primo genitore con quello del secondo.
- **Mutazione** La mutazione applica una variazione casuale all'individuo, nodo per nodo, con probabilità `PMUT_PER_NODE`. Il comportamento dipende dal tipo di nodo:
  - se è un terminale, viene rimpiazzato con un altro terminale casuale;
  - se è un operatore, viene sostituito da un altro operatore aritmetico (ADD, SUB, MUL, DIV).

## 4.4 Ciclo evolutivo

TinyGP utilizza un approccio *steady-state*, in cui la popolazione viene aggiornata individuo per individuo, anziché rigenerare l'intera popolazione ad ogni generazione. Ad ogni generazione:

1. Viene generato un nuovo individuo tramite crossover (90%) o mutazione (10%), partendo dai due individui selezionati dalla popolazione tramite una `TOURNMENT_SELECTION` (sceglie casualmente un piccolo gruppo di individui, confronta le loro fitness e restituisce l'individuo migliore).
2. L'individuo viene valutato tramite la funzione di fitness.
3. Viene selezionato uno degli individui peggiori tramite `TOURNMENT_SELECTION` negativa.
4. L'individuo peggiore viene rimpiazzato dal nuovo individuo.
5. Vengono registrate statistiche quali:
   - fitness media;
   - fitness del migliore;
   - dimensione media degli alberi;
   - miglior individuo corrente in forma infissa.

Il processo continua per un numero prefissato di generazioni oppure fino a quando l'errore diventa trascurabile.

Quando un individuo raggiunge una fitness sufficientemente elevata (errore quasi nullo), l'algoritmo:

- stampa il programma ottimale in notazione leggibile;
- memorizza la funzione evoluta nel file `best_program.txt`, da dove l'algoritmo di classificazione prenderà in input la formula calcolata;
- arresta l'esecuzione anticipatamente.

## 4.5 Esecuzione dell'algoritmo

L'algoritmo viene avviato passando come parametro un file contenente il dataset, nel formato:

```
<num_variabili> <num_costanti> <min_random> <max_random> <num_casi>
<feat1> <feat2> ... <target>
...
```

Il comando di esecuzione è:

```
java Tiny_GP problem.dat
```

```
Generation= 208
AvgFitness= 109.81509459361567
BestFitness= 3.000214621588749
AvgSize= 99.556
BestIndividual=
(((((((((X2 / X4) / X1) - X4) / X1) + ((((1,42 - X1) + X1) * (X4 - ((X2 / X4) * X1))) * (((X3 + X3) /
 / X1) + (X1 / X1)) / X4)) + ((((1,42 / X2) * X1) * ((X4 - ((X1 / X4) / X3)) - (((((1,42 / X2) / X1)
))) / (((X1 + X4) - (X4 - 0,41)) - X4))) - (((X1 / X4) / (X4 / X3)) * X1))
```

```
Generation= 287
AvgFitness= 128.62193643889754
BestFitness= 3.1395445440469306E-7
AvgSize= 145.7192
BestIndividual=
(((((((((X2 + X4) / X1) - X2) / X1) + (((X1 + X1) * (X1 - (X2 + X3))) * (((X1 + 1,42) / (X2 - ((X4 -
(1,42 - X1) + (((X2 / X4) / X4) / X4)) / X3)) + ((((1,42 / X2) * X4) * ((((((((((X2 + X4) / X1) - X2)
X4) + X3))) * (((X1 + X4) / (X4 - ((X3 - ((X2 / X1) / X3)) / X3))) * X1))) * (((1,42 - X1) + (((X2 /
X2) * X4) * ((((X2 + 1,42) - (X4 - X4)) - X1) - (((((1,42 / X2) / X4) - (X3 - ((X2 + X4) * X4))) / X4
) - X1))) - X4) - (((((1,42 / X2) / X4) - (X3 - ((X2 + X4) * X4))) / X4) / X3))) / (((X1 + X4) - (X4

PROBLEM_SOLVED

Stampo su file il best program
```

Figura 4.1: Esempio dell'output in fase di generazione

# Capitolo 5

# Classificazione e Valutazione dei Risultati

In questo capitolo viene descritto come il modello di Programmazione Genetica evoluto viene applicato al dataset di test per classificare le immagini e come vengono valutati i risultati. Il modello utilizza il miglior individuo trovato durante l'evoluzione, salvato nel file `best_program.txt`, per generare le previsioni.

## 5.1 Classificazione di un singolo esempio

La funzione `load_gp_formula` legge il file contenente la formula del miglior individuo GP calcolata dal programma Java TinyGP.
La classificazione di un singolo esempio avviene tramite la funzione `classify`, che:

- prende in input le quattro feature normalizzate dell'immagine: `X1, X2, X3, X4`;
- valuta la formula GP tramite `eval` in Python;
- applica la funzione sigmoide per ottenere una probabilità compresa tra 0 e 1;
- interpreta la probabilità come:
  - $\geq 0.5$: immagine classificata come *quadrato*;
  - $< 0.5$: immagine classificata come *cerchio*.

La funzione gestisce eventuali errori numerici (NaN o infiniti), sostituendo il risultato con 0.5 come valore di default.

**Algorithm 4** Classificazione di un singolo esempio tramite GP

**Require:** Feature $X_1, X_2, X_3, X_4$; formula GP $gp_formula$
**Ensure:** Classe predetta

```
Valuta la formula GP usando i valori X1, X2, X3, X4
if si verifica un errore o il risultato non è numerico then
    result 0.0
end if
Applica la funzione sigmoide al risultato: output = 1/(1+e^(-result))
if output ≥ 0.5 then Label predetta = "quadrato" else Label predetta = "cerchio"
return output
```

## 5.2 Classificazione del dataset di test

La funzione `classify_images` esegue la classificazione per tutte le immagini di una cartella:

1. Legge tutte le immagini nella cartella specificata.
2. Per ciascuna immagine applica la funzione `classify_images` riportata con lo pseudocodice prima, assegnando la label predetta in base alla probabilità di ogni immagine. Da notare che l'estrazione delle features avviene nella stessa maniera descritta nell'algoritmo 3 della sezione 3.2.
3. Memorizza i risultati in un `DataFrame` (usando la libreria `Pandas`) e successivamente in `CSV` `classification_results.csv`), utile per andare a calcolare l'accuracy in un secondo momento.

## 5.3 Valutazione delle prestazioni

Per valutare l'efficacia del modello GP si confrontano le label predette con le label reali delle immagini di test:

- Viene creata la colonna `true_label` basata sul nome del file, distinguendo tra immagini di quadrati e cerchi. Questa nuova colonna viene aggiunta al dataframe `classification_results.csv` in modo tale che è possibile avere per ogni immagine le 4 feature estratte, la laber reale e la label predetta.
- Si calcola l'accuratezza come:

$$\text{Accuracy} = \frac{\text{n° di immagini correttamente classificate}}{\text{n° totale di immagini}} \cdot 100$$

```
==========================================================================================
Formula GP caricata:
((((((X2 / (((X2 / (X3 + (X1 + ((X2 * X4) - ((X2 + X4) - X4))))) - X1) + (X1 / ((X1 * X4) - ((X2 + X4)
 - X3))))) - X3) - ((1.85 / X3) / -0.06)) + (X2 + (X4 - X4))) / ((((1.85 * X4) - ((1.85 * X1) + -0.06)
) - 0.85) / (X3 + ((X2 / ((X3 / X2) / X2)) / X4)))) / ((X2 * ((1.85 * X4) - (X2 - -0.06))) - (X2 - X1)
))
==========================================================================================

Risultati salvati in classification_results.csv

================================================
Accuratezza complessiva: 96.33% (289/300)
================================================
```

(a) Output classificatore

```
filename,area,perimeter,aspect_ratio,extent,prob,label_predicted,true_label
circle_000.png,0.38,0.53,0.56,0.53,0.0,cerchio,cerchio
circle_001.png,0.38,0.53,0.56,0.53,0.0,cerchio,cerchio
circle_002.png,0.01,0.03,0.56,0.37,0.0,cerchio,cerchio
circle_003.png,0.08,0.17,0.56,0.45,0.0,cerchio,cerchio
circle_004.png,0.45,0.6,0.36,0.64,0.0,cerchio,cerchio
circle_005.png,0.25,0.4,0.56,0.5,0.0,cerchio,cerchio
circle_006.png,0.84,0.9,0.56,0.56,0.0,cerchio,cerchio
```

(b) Dataframe

# Capitolo 6

# Conclusioni

In questo lavoro è stato sviluppato un sistema di classificazione in grado di distinguere tra immagini di cerchi e quadrati mediante un approccio basato sulla *Genetic Programming*. L'intero processo è stato implementato utilizzando il **TinyGP**, una versione compatta della GP che permette di evolvere programmi simbolici in maniera semplice e controllata. TinyGP ha costituito il "core" dell'algoritmo evolutivo, gestendo tutte le operazioni fondamentali quali generazione della popolazione, valutazione della fitness, selezione tramite torneo, crossover e mutazione.

TinyGP ha evoluto progressivamente programmi capaci di combinare queste feature per minimizzare l'errore di classificazione, utilizzando una funzione di fitness basata sulla distanza tra l'output del programma e la label reale.

Il programma migliore ottenuto è stato successivamente valutato sul test set, dopo una fase di normalizzazione delle feature e applicazione della funzione sigmoide per trasformare l'output della GP in una probabilità. La valutazione ha mostrato un'accuratezza complessiva sopra al **90%**, in base a che ti tipologia di dataset di test veniva usato. Indicando una buona capacità di generalizzazione anche su immagini non viste durante l'addestramento.

Si può concludere quindi che il **TinyGP** e in generale la programmazione evolutiva siano in grado di produrre modelli interpretabili e funzionali, rappresentando una valida alternativa ai metodi di apprendimento più complessi. Possibili estensioni future includono l'uso di feature più avanzate, dataset più complessi, o una classificazione multi-classe per andare a fare il labeling di più tipologie di figure.

Il progetto presenta alcuni limiti principali. Le feature utilizzate sono semplici e descrivono solo in parte la forma delle figure, rendendo il modello sensibile a rumore, deformazioni o posizioni delle figure non previste. Inoltre, l'uso di TinyGP, pur vantaggioso per semplicità, limita la complessità delle soluzioni evolvibili perchè adotta operatori evolutivi essenziali e una rappresentazione ad albero semplice portando una dipendenza dalla scelta iniziale dei parametri evolutivi (dimensione della popolazione, profondità massima degli alberi, probabilità di crossover e mutazione).